Case Study
on
Unified Payments Interface (UPI): A Large-Scale Software Engineering System

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

Computer Science Engineering

Submitted by:

Harkamal Takhi

UID: 24BCS11583

Section & Group: 24BCS_SAP-611-A

Under the Guidance of
Dr. Anuj Kumar Jain

INSTITUTE – UIE
Computer Science and Engineering



Index

S. No.	Topic	Page No.
1.	Abstract	3
2.	Introduction	3
3.	Objective	3
4.	SDLC Model Used	4
5.	Requirement Analysis	5
6.	System Architecture	6
7.	Data Flow Diagram	7
8.	Use Case Diagram	9
9.	Entity Relationship Diagram	10
10.	Data Dictionary	11
11.	Conclusion	12

Abstract

The Unified Payments Interface (UPI) is a real-time digital payment system that enables instant bank-to-bank transactions through mobile applications. Developed by the National Payments Corporation of India (NPCI), UPI has transformed the digital payments landscape by offering a secure, scalable, and interoperable platform. This case study analyzes UPI from a software engineering perspective, focusing on its system architecture, Software Development Life Cycle (SDLC) model, requirement analysis, data flow modeling, database design, and security engineering mechanisms. The study further evaluates functional and non-functional requirements, risk mitigation strategies, and system performance characteristics. By applying hybrid SDLC practices, distributed system design, and layered security architecture, UPI demonstrates how large-scale financial systems can achieve high availability, reliability, and scalability. This report provides a structured engineering analysis of UPI to understand its design principles and implementation strategies.

Introduction

Digital payment systems have significantly evolved over the last decade. Traditional banking methods required physical presence, complex account details, and time-consuming processing. To simplify digital transactions and promote financial inclusion, NPCI introduced the Unified Payments Interface (UPI).

UPI enables users to transfer funds instantly using a Virtual Payment Address (VPA) instead of bank account numbers. It supports peer-to-peer (P2P) and peer-to-merchant (P2M) transactions with 24x7 availability.

From a software engineering perspective, UPI represents:

1. A large-scale distributed system
2. A secure financial transaction engine
3. An API-driven architecture
4. A high-availability infrastructure

The success of UPI demonstrates the importance of scalability, security engineering, and proper SDLC implementation in mission-critical systems.

Objective

The objectives of this case study are:

- To analyse the architecture of UPI.
- To examine the SDLC model used in its development.
- To identify functional and non-functional requirements.
- To model system behaviour using Data Flow Diagrams.
- To design Use Case and Entity Relationship diagrams.
- To define database components using a Data Dictionary.

- To evaluate system security and risk mitigation strategies.
- To understand scalability and performance engineering techniques.

SDLC Model Used

UPI follows a Hybrid Software Development Life Cycle model combining structured planning with Agile and DevOps practices.

Requirement Analysis Phase

In this phase, stakeholders such as RBI, NPCI, banks, and fintech partners identified the need for:

- Real-time transactions
- High security
- Interoperability among banks
- Mobile-first design
- Regulatory compliance

System Design Phase

The system architecture was designed using:

- Central switching model
- API-based communication
- Encrypted data transfer
- Distributed servers
- Redundant data centres.

Implementation Phase

Developers implemented:

- Secure APIs
- Authentication modules
- Transaction processing logic
- Fraud detection systems
- Logging and monitoring mechanisms

Testing Phase

Testing included:

- Unit Testing
- Integration Testing
- System Testing
- Load Testing
- Security Testing
- Failover Testing

Deployment Phase

Deployment was executed gradually through:

- Bank onboarding
- Continuous Integration / Continuous Deployment (CI/CD)
- Monitoring dashboards

Maintenance Phase

Continuous updates are provided for:

- Security patches
- Feature upgrades
- Fraud detection improvements
- Regulatory updates

Requirement Analysis

A. Functional Requirements

Functional requirements describe what the system must do.

1. The system shall allow user registration.
2. The system shall allow linking of bank accounts.
3. The system shall generate a unique UPI ID.
4. The system shall enable money transfer between accounts.
5. The system shall authenticate users using UPI PIN.

6. The system shall generate transaction confirmation.
7. The system shall maintain transaction history.
8. The system shall allow QR code-based payments.
9. The system shall notify users about transaction status.
10. The system shall handle transaction failures.

B. Non-Functional Requirements

Non-functional requirements describe system quality attributes.

Performance

Transactions must complete within 5 seconds.

Security

System must implement:

- Two-factor authentication
- End-to-end encryption
- Device binding

Scalability

System must support millions of daily transactions.

Availability

System must operate 24x7 with minimal downtime.

Reliability

System must ensure data consistency and no transaction loss.

Maintainability

System must allow easy updates and modular modifications.

Usability

Interface must be simple and user-friendly.

System Architecture

UPI follows a layered and distributed architecture.

Major Components:

1. User Mobile Application
2. Payment Service Provider (PSP) Server
3. NPCI Central Switch
4. Issuer Bank
5. Acquirer Bank
6. Settlement System

Architectural Characteristics:

- Central switching architecture
- API-driven communication
- Distributed databases
- Redundant servers
- Load balancing

The NPCI central switch acts as an intermediary routing system between banks.

Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)

At this level, the entire UPI system is represented as a single process.

External Entities:

- User
- Issuer Bank
- Acquirer Bank

Data Flows:

- Payment Request
- Authentication Data
- Transaction Response

- Confirmation

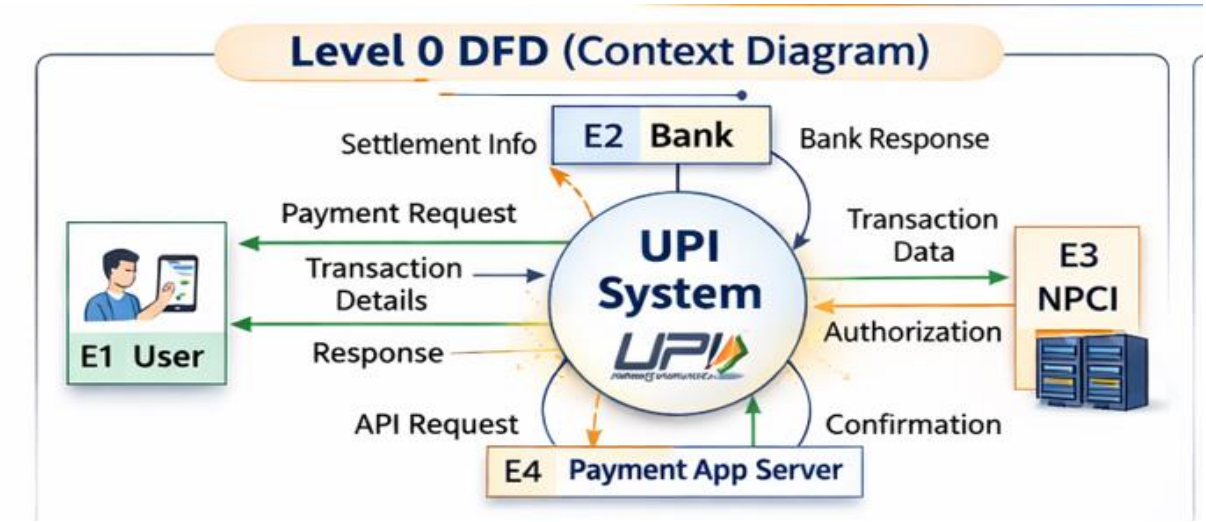


Fig.: Level 0 DFD

Level 1 DFD

The system is broken into sub-processes:

1. Authenticate User
2. Validate Bank Details
3. Process Transaction
4. Update Transaction Database
5. Send Confirmation

Data Stores

1. User Database
2. Bank Account Database
3. Transaction Database
4. Fraud Detection Logs
5. Audit Logs

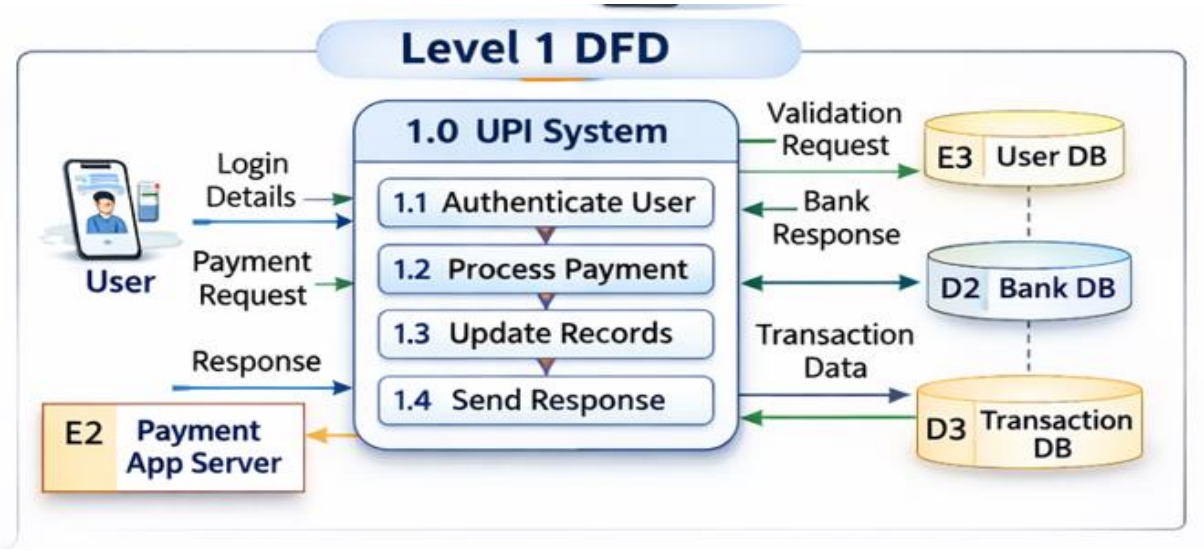


Fig.: Level 1 DFD

Use Case Diagram

Actors:

1. User
2. Bank
3. NPCI System
4. Administrator

Use Cases:

- Register
- Login
- Link Bank Account
- Send Money
- Receive Money
- Check Balance
- View Transaction History
- Generate QR Code
- Monitor Fraud

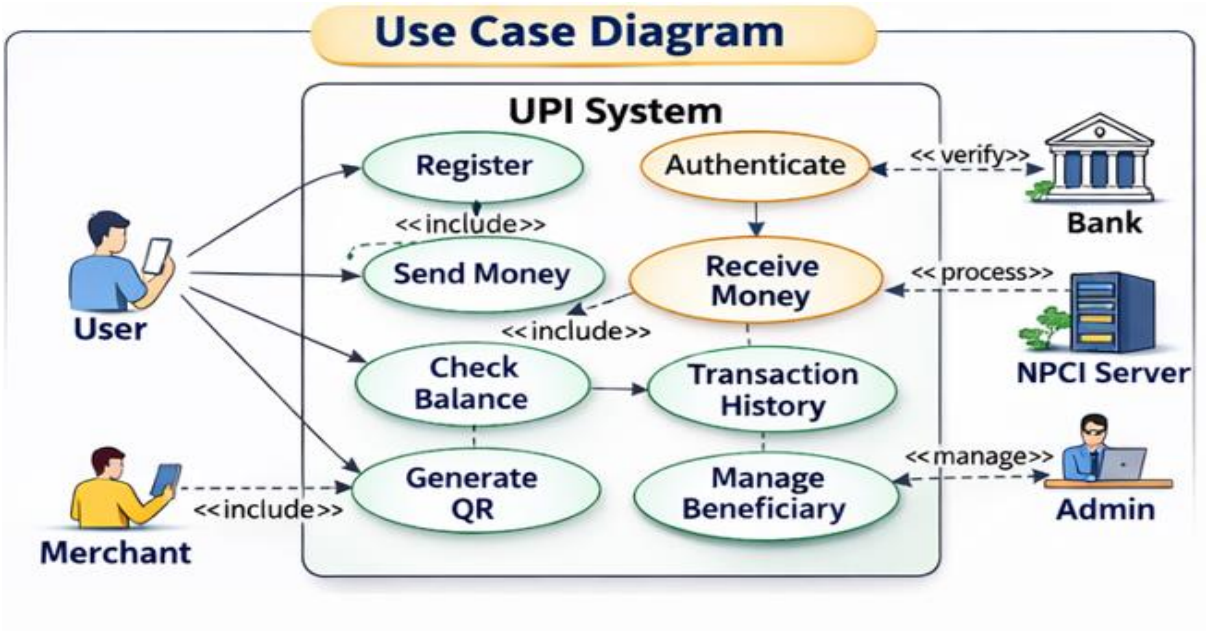


Fig.: Use Case Diagram

Entity Relationship Diagram (ER Diagram)

Entities:

User (User_ID, Name, Mobile_No)
BankAccount (Account_ID, Bank_Name, Balance)
Transaction (Transaction_ID, Amount, Timestamp)
UPI_ID (UPI_ID, User_ID)
Merchant (Merchant_ID, Business_Name)

Relationships:

- One User has many BankAccounts.
- One User has one UPI_ID.
- One BankAccount has many Transactions.
- One Merchant receives many Transactions.

Primary Keys:

- User_ID
- Account_ID
- Transaction_ID

Foreign Keys:

- User_ID in BankAccount

- Account_ID in Transaction

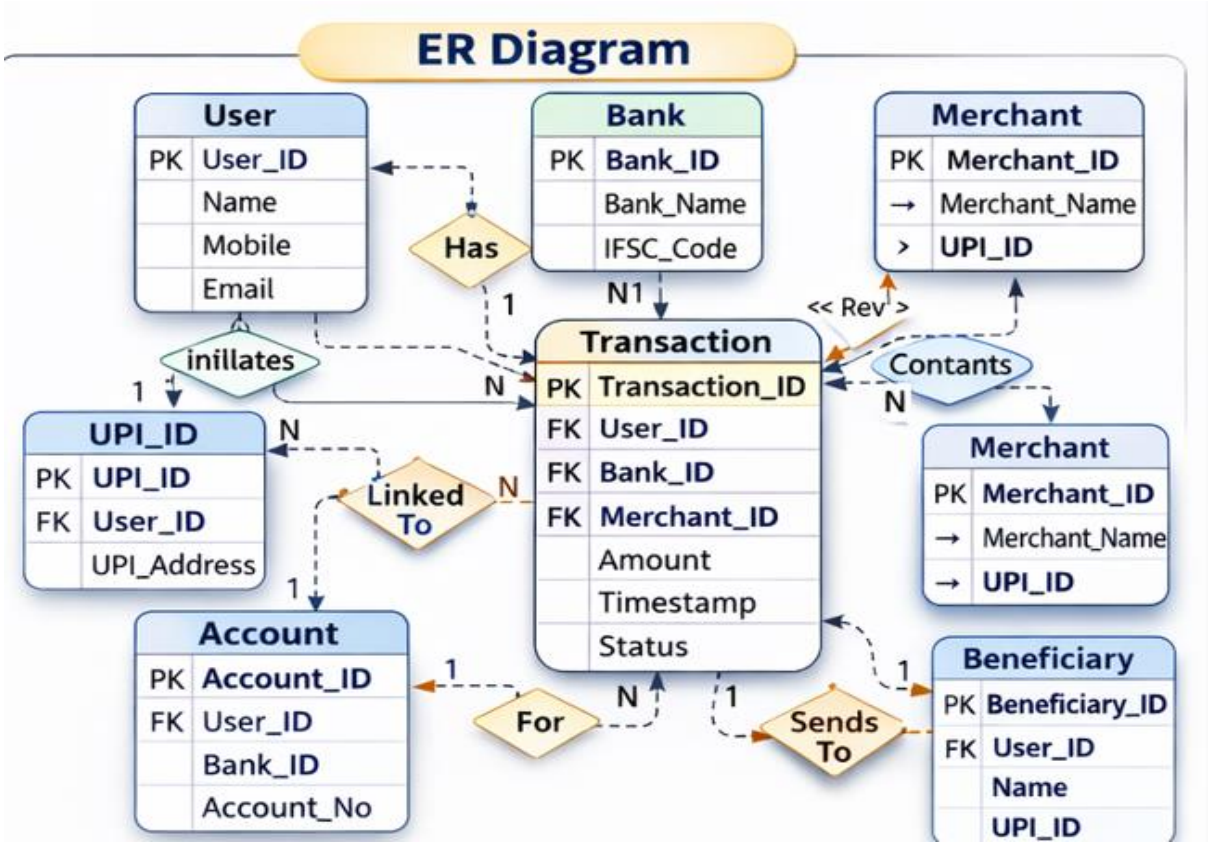


Fig.: ER Diagram

Data Dictionary

A Data Dictionary defines each attribute.

Field Name	Data Type	Description
User_ID	Integer	Unique user identifier
UPI_ID	Varchar	Virtual Payment Address
Account_ID	Integer	Bank account identifier
Transaction_ID	Integer	Unique transaction number
Amount	Decimal	Money transferred
Timestamp	DateTime	Data & Time of transaction
Status	Varchar	Success/Failed
PIN	Encrypted	User authentication PIN

Conclusion

The Unified Payments Interface represents a successful implementation of large-scale software engineering principles in the financial sector. By adopting a hybrid SDLC model, layered architecture, strong security mechanisms, and distributed scalability strategies, UPI ensures reliability, availability, and performance. The system effectively addresses functional requirements such as real-time transactions while meeting non-functional requirements such as security, scalability, and usability. Through proper requirement analysis, structured design modeling, and continuous maintenance, UPI demonstrates how engineering methodologies can support national-level digital infrastructure. This case study highlights the importance of combining system design, data modeling, security engineering, and risk mitigation in building mission-critical applications.